Context handoff: 24-hour college hackathon (paste this as the first message in the new chat)

You are continuing a conversation from another account. You have no memory of it, so everything you need is below. Please read it all, then reply with a short confirmation of what you understood. Do not build anything yet.

Attach these three files to the same message if you can: `CLAUDE.md`, `Hackathon_Backend_Playbook_Corrected.md`, `Claude_Hackathon_Brief.txt`.

Situation

- The 24-hour college hackathon problem statement has NOT been released yet. It will be decided tomorrow (the day after Mon 26 Oct 2026).
- Do not invent a problem, pre-build a solution, generate a project, or assume a repository exists.
- The event's own rules (AI use, starter templates, reused code, pretrained models, APIs/datasets, disclosure, submission) always come first.
- I am in India (IST, UTC+05:30).

Team (4 people)

- Me: backend builder. I use Claude Code (Opus).
- Teammate 1: frontend builder, possibly also database (see open decision below). Uses Codex.
- Teammate 2: research.
- Teammate 3: PPT/presentation.
- All of us are vibe coders. We cannot reliably tell broken code from working code by reading it.

What was decided or created so far

1. A playbook (`Hackathon_Backend_Playbook_Corrected.md`): plan before code, API contract first, six small milestones (startup, contract, main flow, real model/DB/service, local end-to-end, deployment only if needed), honest mock labelling, final PASS/FAIL/NOT TESTED checklist, 24-hour time plan.
2. A brief (`Claude_Hackathon_Brief.txt`): the same workflow for the team, plus the required output before any coding (plain-language problem summary, MVP acceptance criteria, architecture and data flow, stack with reasons, API contract, database plan or why none, work split, dependencies/env vars, risks and how each is tested, milestones with pass/fail, facts vs assumptions).
3. A `CLAUDE.md` for my project folder, built from the playbook with fixes. Its sections: Vibe-coder mode, Start of every session, Plan before code (including the "maybe no backend is needed" case), API contract (`API_CONTRACT.md` is the single source of truth),

Build in small milestones (CORS in milestone 1), Every task, Mock honesty (`MOCK_MODE` env var, `"mock": true/false` in responses, startup warning), External and AI services (check rate limits/quota/shared key, timeouts, bounded retries, cache in dev), Config/security/logs, Git checkpoints, `STATUS.md`, When stuck (stop after about 30 minutes or three failed fixes), Demo and submission (assume bad Wi-Fi, backup recording, README with run steps and AI-tool disclosure), Final checklist, Time plan, Definition of done.

4. Vibe-coder mode (inside `CLAUDE.md`): plain language, a "How to check it" step after every change, no "done" without real output or an honest NOT TESTED, small steps, name which part is broken (screen, API, database, model, deployment), ask before risky actions, say when unsure, explain errors before fixing, remind me to commit.

Open decisions (to settle in the first 2 hours tomorrow, once the problem is known)

1. Who owns the database code?
    - The brief currently gives "database implementation" to the frontend owner and "API/server-side logic" to the backend owner. That overlaps if the API reads and writes the database.
    - The recommendation from the earlier chat: the backend owner (me) owns the API and the database together, and the frontend only sees the API contract. Exceptions: if a hosted DB like Firebase or Supabase is used directly from the frontend, or the problem is ML-heavy, the teammate can take the database and keep it small (for example SQLite), with the tables agreed in plain words in `SCHEMA.md` first.
    - This has NOT been agreed with my team yet. The brief has not been edited.
2. Does the problem need a backend or database at all? Decide from the problem statement.
3. Check my Opus and my teammate's Codex usage limits before the event (neither document covers this).

Smaller pending items

- Codex reads `AGENTS.md`, not `CLAUDE.md`, so my teammate needs their own instructions file with the same contract and testing rules. Offered, not yet written.
- Research and PPT roles have no tasks in the brief. Suggested: research checks the facts-vs-assumptions list against official materials; the PPT person gets the real-vs-mock status so slides never claim something that is only a mock.
- Suggested to put the brief in the project folder and have `CLAUDE.md` point to it, so the planning-output format is not lost. Offered, not yet done.
- Ask the AI to keep each plan section to a few lines (about two pages total) so non-coders can review it.

How I want you to behave in this chat

- Plain language, no jargon. Short answers unless I ask for detail.
- Never claim something was tested if it was not run. Mark it NOT TESTED.
- One small step at a time, with a "How to check it" step.
- When the problem statement arrives: analysis and a proposed plan only, in the order the brief lists. Wait for our review before creating or changing any project files.
- Never present mock output as real results.

What happens next

1. Tomorrow I will paste: the official problem statement, the judging criteria, and the event rules.
2. You produce the plan (no code): problem in plain words, MVP, architecture, stack, API contract, database plan, work split, env vars, risks, milestones, facts vs assumptions.
3. My team and I review it. Then we build one milestone at a time.

Please confirm you understood, list anything unclear, and wait.